AIと機密性の高い推論を担当- ネットアップのAIとProtopia Image Transformation

NetApp Solutions

NetApp
May 31, 2022

This PDF was generated from https://docs.netapp.com/ja-jp/netapp-solutions/ai/ai-protopia-overview.html on May 31, 2022. Always check docs.netapp.com for the latest.

目次

AIと機密性の高い推論を担当-ネットアップのAIとProtopia Image Transformation 1
TR-4928：『Responsible AI and Confidential Inferencing - NetApp AI with Protopia Image Transformation』 1
解決策エリア 3
テクノロジの概要 4
テストと検証の計画 9
設定をテストします 9
手順をテストします 10
推論の精度の比較 24
難読化の速度 25
まとめ 25
追加情報 、確認応答、およびバージョン履歴の参照先 26

AIと機密性の高い推論を担当-ネットアップのAIとProtopia Image Transformation

TR-4928：『Responsible AI and Confidential Inferencing - NetApp AI with Protopia Image Transformation』

Sathish Thyagarajan氏、Michael Oglesby氏、NetApp Byung Hoon Ahn氏、Jennifer Cwagenberg氏、Protopia氏

画像の撮影や画像処理の出現とのコミュニケーションには、視覚的な解釈が不可欠です。デジタル画像処理における人工知能（AI）は、がんやその他の疾患識別のための医療分野などの新たなビジネスチャンスをもたらします。また、地球空間での視覚分析による環境ハザード調査、パターン認識、犯罪と闘うためのビデオ処理などにも活用できます。しかし、この機会には特別な責任も伴います。

AIを導入する意思決定が増えるほど、データのプライバシー、セキュリティ、法律、倫理、規制に関する問題に関連するリスクを受け入れることができます。責任あるAIは、大企業のAIに欠かせない信頼とガバナンスを企業や政府機関が構築できるようにするための実践を可能にします。本ドキュメントでは、ネットアップが3つの異なるシナリオで検証したAI推論解決策 について説明します。この検証では、Protopiaデータ難読化ソフトウェアとネットアップのデータ管理テクノロジを併用して、機密データをプライベート化し、リスクと倫理的な懸念を軽減します。

毎日何百万もの画像が生成され、消費者とビジネスエンティティの両方がさまざまなデジタルデバイスを使用します。その結果、データとコンピューティングのワークロードが急増し、企業はクラウドコンピューティングプラットフォームを利用して、拡張性と効率性を高めることになります。一方、画像データに含まれる機密情報に関するプライバシーの懸念は、パブリッククラウドへの転送によって生じます。画像処理AIシステムの導入における主な障壁は、セキュリティとプライバシーの欠如です。

また、もあります "イレイジャーコーディングの権利" GDPRでは、組織がすべての個人データを消去するように要求する権利が個人に与えられます。また、もあります "プライバシー法"は、公正な情報慣行のコードを確立します。写真などのデジタル画像は、GDPRに基づく個人データを構成し、データの収集、処理、消去の方法を規定します。これを怠るとGDPRに準拠できず、組織に深刻な損害を与える可能性があるコンプライアンス違反に対する多額の罰金が科せられる可能性があります。プライバシーに関する原則は、機械学習（ML）モデルとディープラーニング（DL）モデルの予測において公平性を確保し、プライバシーや規制へのコンプライアンス違反に伴うリスクを軽減する、責任あるAIを実装するためのバックボーンとなっています。

このドキュメントでは、プライバシーの保護と責任あるAI解決策 の導入に関連する画像難読化機能を使用した、または使用しない、3つの異なるシナリオにおける検証済み設計解決策 について説明します。

- シナリオ**1.** Jupyterノートブック内でのオンデマンド推論。
- シナリオ**2.** Kubernetesでのバッチ推論。
- シナリオ**3.** NVIDIA * Triton推論サーバ。

この解決策 では、フェース検出データセットとベンチマーク（FDDB）を使用します。これは、フェース検出の問題を調査するために設計されたフェース領域のデータセットで、PyTorch機械学習フレームと組み合わせて、フェースボックスを実装するためのフレームワークです。このデータセットには、さまざまな解像度の2845枚の画像セットに5171個の面の注釈が含まれています。さらに、このテクニカルレポートでは、この解決策 を適用できる状況において、ネットアップのお客様やフィールドエンジニアから収集した解決策 の領域と関連するユースケースを紹介します。

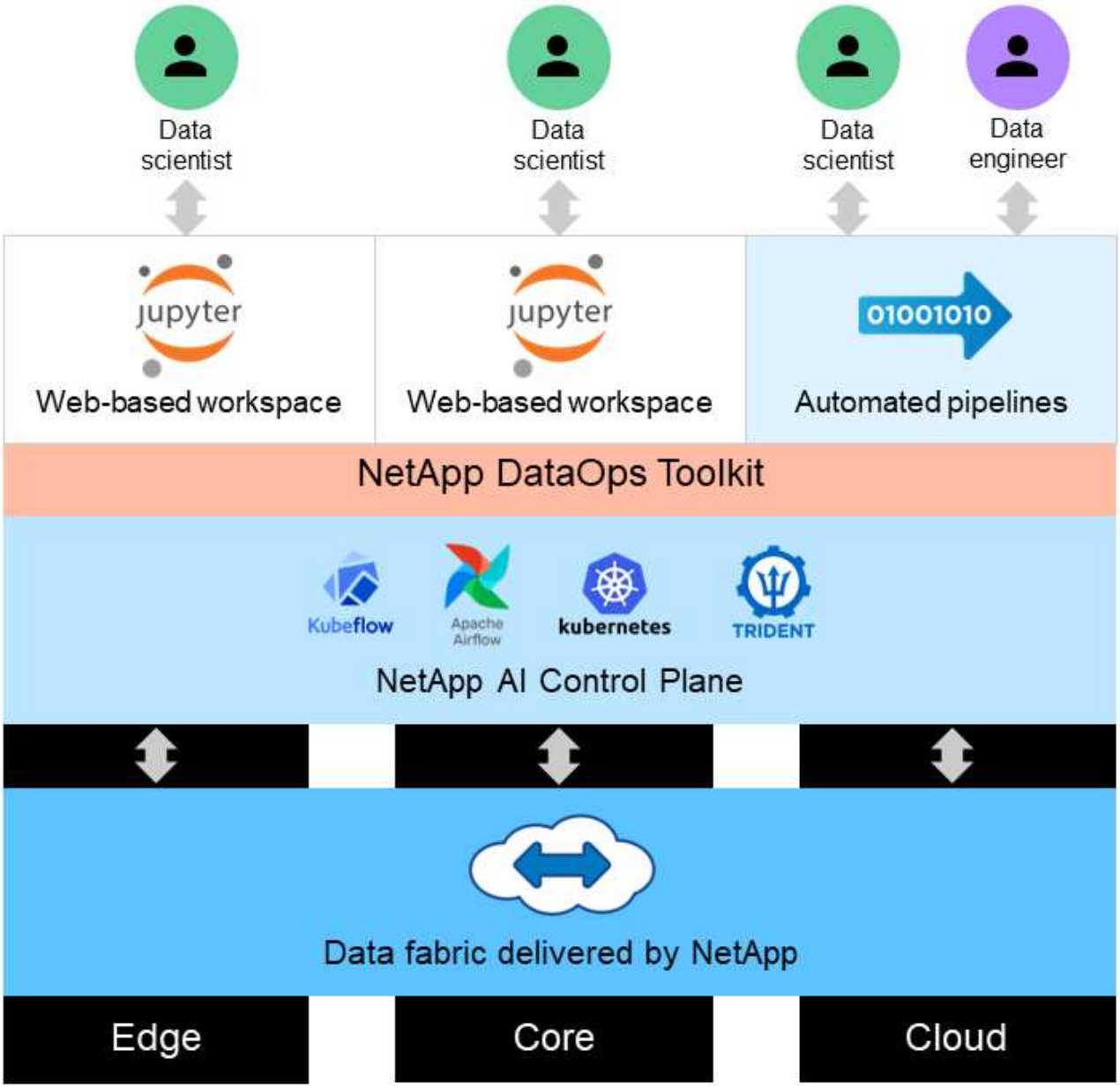
対象読者

このテクニカルレポートは、次のような方を対象としています。

- 公共スペースでの顔画像処理に関するデータ保護やプライバシーの問題に対処するために、責任あるAIの設計と導入を希望するビジネスリーダーおよびエンタープライズアーキテクト。
- データサイエンティスト、データエンジニア、AI /機械学習（ML）研究者、AI / MLシステム開発者は、プライバシーの保護と維持を目指します。
- GDPR、CCPA、国防総省（DoD）や政府機関のプライバシー法などの規制基準に準拠するAI / MLモデルおよびアプリケーション向けのデータ難読化ソリューションを設計するエンタープライズアーキテクト。
- データサイエンティストとAIエンジニアは、機密情報を保護するディープラーニング（DL）モデルとAI / ML / DL推論モデルを効率的に導入する方法を探しています。
- エッジ推論モデルの導入と管理を担当するエッジデバイスマネージャとエッジサーバ管理者。

解決策アーキテクチャ

この解決策 は、GPUの処理能力と従来のCPUを併用することで、大規模なデータセットにおけるAIワークロードのリアルタイムおよびバッチ推論を処理するように設計されています。この検証では、MLのプライバシー保護推論と、責任のあるAI導入を求めている組織に必要な最適なデータ管理が実証されています。この解決策 は、エッジコンピューティングとクラウドコンピューティングを連携させる単一ノードまたはマルチノードのKubernetesプラットフォームに適したアーキテクチャを提供します。このプラットフォームは、オンプレミスのコアソフトウェアであるNetApp ONTAP AI、NetApp DataOpsツールキット、およびJupyter LabとCLIインターフェイスを使用したProtopia難読化ソフトウェアと連携します。次の図は、ネットアップのDataOpsツールキットとProtopiaを使用したデータファブリックの論理アーキテクチャの概要を示しています。



Protopia難読化ソフトウェアは、NetApp DataOpsツールキットの上でシームレスに実行され、ストレージサーバから離れる前にデータを変換します。

"次の例：解決策 Areas"

解決策エリア

"前へ：概要。"

デジタル画像処理には多くの利点があり、多くの組織が視覚表現に関連するデータを最大限に活用できるようになっています。このネットアップとProtopia解決策 は、ML / DLのライフサイクル全体にわたってAI / MLデータを保護し、民営化するための、独自のAI推論設計を提供しています。お客様は機密データの所有権を維持し、プライバシーに関する懸念を解消して、拡張性と効率性を高めるためにパブリッククラウドまたはハイブリッドクラウドの導入モデルを使用できます。また、エッジでAI推論を導入することもできます。

環境インテリジェンス

業界は、環境上の危険がある領域で地理空間分析を活用する方法が数多くあります。政府や公共事業部は、公衆衛生や天候に関する実用的な洞察を得ることができ、パンデミックや野生火災などの自然災害時の国民へのアドバイスを向上させることができます。たとえば、空港や病院などの公共スペースで新型コロナウイルス感染症の患者を特定する際に、該当する個人のプライバシーを侵害することなく、必要な安全対策を講じるために各当局および近隣の公衆に警告を出すことができます。

エッジデバイスのウェアラブル

軍事演習や戦場では、エッジでのAI推論をウェアラブルデバイスとして使用することで、兵士の健康状態を追跡し、運転者の行動を監視し、軍車に接近する際の安全および関連するリスクについて当局に警告しながら、兵士のプライバシーを守り、保護することができます。軍事の未来は、Internet of Battlefield Things（IOBT）とInternet of Military Things（IANMT）でハイテク化を進めています。この製品は、兵士が敵を特定し、高速エッジコンピューティングを使用して戦闘でより良いパフォーマンスを発揮するのに役立ちます。ドローンやウェアラブル・ギアなどのエッジデバイスから収集した視覚的なデータを保護し、保護することは、ハッカーや敵をベイに維持するために不可欠です。

非燃焼型避難作業

非戦闘員避難行動（Neos）は、米国市民および国民、DoD民間人、および指定された人（HN）および第3国国民（TCN）の避難を支援するために、国防総省によって実施されます。所定の管理制御では、主に手動による退避対象者スクリーニングプロセスが使用されます。ただし、高度に自動化されたAI / MLツールとAI / MLビデオ難読化テクノロジを組み合わせることで、避難先の識別、退避対象の追跡、脅威スクリーニングの精度、セキュリティ、速度を向上させることができます。

AI / ML分析のクラウド移行

エンタープライズのお客様は従来、AI / MLモデルのトレーニングを受け、オンプレミスで導入してきました。スケールメリットと効率性の理由から、AI / ML機能をパブリッククラウド、ハイブリッドクラウド、マルチクラウド環境に移行するお客様が増えています。ただし、他のインフラにどのようなデータを公開できるかによって制限されます。ネットアップのソリューションは、に求められるあらゆるサイバーセキュリティの脅威に対処します "データ保護" また、セキュリティ評価を行い、Protopiaのデータ変換と組み合わせることで、画像処理AI / MLワークロードをクラウドに移行する際のリスクを最小限に抑えることができます。

他の業界のエッジコンピューティングとAI推論のその他のユースケースについては、を参照してください "TR-4886『エッジでのAI推論』" NetApp AIブログ、 "インテリジェンスとプライバシー"。

"次のステップ：テクノロジの概要"

テクノロジの概要

"前のページ：解決策 Areas"

プロトピア

Protopia AIは、今日の市場における機密性の高い推論のための、目立たないソフトウェア型解決策 を提供します。Protopia解決策 は、機密情報の漏洩を最小限に抑えることで、推論サービスに対する比類のない保護機能を提供します。AIには、データレコード内の情報のみが提供されます。この情報は、実際にはタスクを実行するために不可欠であり、それ以上何も必要ありません。ほとんどの推論タスクでは、すべてのデータレコードに存在するすべての情報が使用されるわけではありません。画像、音声、ビデオ、構造化された表形式デー

タのどれをAIが消費しているかにかかわらず、Protopiaは推論サービスが必要としているものだけを提供します。特許取得済みのコアテクノロジーは、数学的にキュレーションされたノイズを使用してデータを変革し、特定のMLサービスでは必要のない情報を蓄積します。この解決策 はデータをマスクするのではなく、キュレーションされたランダムなノイズを使用してデータ表現を変更します。

Protopia解決策 は、モデルの機能に関して、入力フィーチャースペースの関連情報を保持する勾配ベースの摂動最大化方法としてリプレゼンテーションを変更する問題を計算します。このディスカバリプロセスは、MLモデルのトレーニング終了時に微調整パスとして実行されます。このパスによって確率分布のセットが自動的に生成されると、オーバーヘッドが低いデータ変換によって、これらの分布から生成されたノイズサンプルがデータに適用され、これが難読化されてから推論モデルに渡されます。

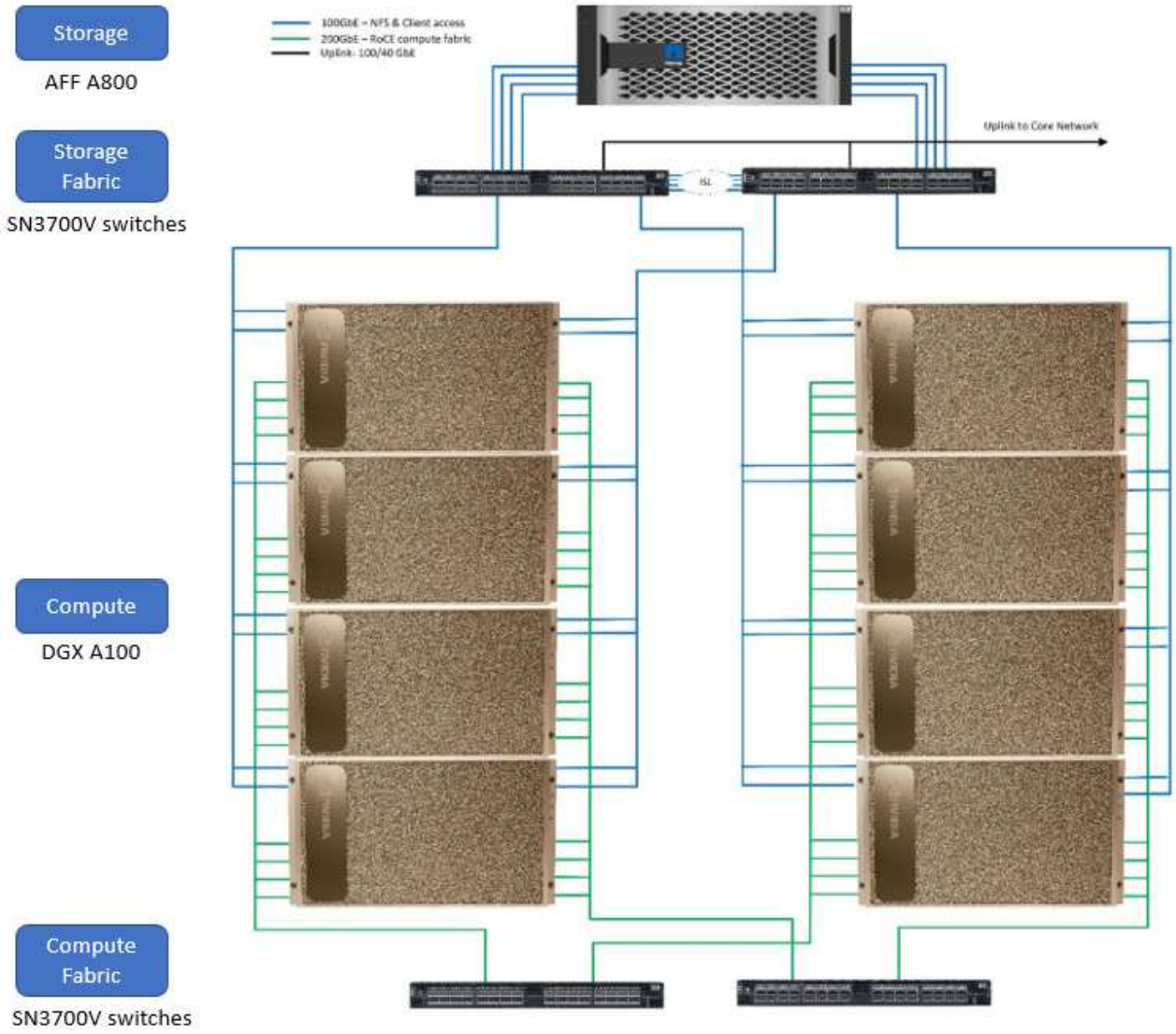
NetApp ONTAP AI

DGX A100システムとネットアップのクラウド対応ストレージシステムを基盤とするNetApp ONTAP AIリファレンスアーキテクチャは、ネットアップとNVIDIAによって開発、検証されました。IT 組織には、次のようなメリットをもたらすアーキテクチャが提供されます。

- 設計の複雑さを解消
- コンピューティングとストレージを個別に拡張できます
- 小規模構成から始めて、シームレスに拡張できます
- は、さまざまなパフォーマンスとに対応するさまざまなストレージオプションを提供します コストポイント

ONTAP AIは、DGX A100システムとNetApp AFF A800ストレージシステムを最先端のネットワークと緊密に統合します。ONTAP AIは、設計の複雑さと推測に頼らず、AI導入を簡易化します。小規模構成から始めて、システムを停止することなく拡張でき、エッジからコア、クラウドまで、データをインテリジェントに管理できます。

次の図は、DGX A100システムを使用したONTAP AIソリューションファミリーのいくつかのバリエーションを示しています。AFF A800システムのパフォーマンスは、最大8台のDGX A100システムで検証されます。ONTAP クラスタにストレージコントローラペアを追加することで、アーキテクチャを複数のラックに拡張して、パフォーマンスがリニアに向上したDGX A100システムとペタバイト規模のストレージ容量をサポートできます。このアプローチにより、使用されるDLモデルのサイズと必要なパフォーマンス指標に基づいて、コンピューティングとストレージの比率を個別に変更できる柔軟性が得られます。



追加情報 About ONTAP AIについては、を参照してください "NVA-1153：NVIDIA DGX A100システムとMellanox Spectrumイーサネットスイッチを搭載したNetApp ONTAP AI"

NetApp ONTAP

ネットアップが提供する最新世代のストレージ管理ソフトウェアONTAP 9.11を使用すれば、インフラを刷新し、クラウド対応データセンターに移行できます。ONTAP は、業界をリードするデータ管理機能を活用して、データの格納場所に関係なく、単一のツールセットでデータの管理と保護を実現します。エッジ、コア、クラウドなど、必要な場所に自由にデータを移動することもできます。ONTAP 9.11には、データ管理を簡易化し、重要なデータを高速化、保護し、ハイブリッドクラウドアーキテクチャ全体で次世代インフラ機能を実現するためのさまざまな機能が搭載されています。

NetApp DataOps ツールキット

NetApp DataOpsツールキットはPythonライブラリで、開発者、データサイエンティスト、DevOpsエンジニア、データエンジニアは、新しいデータボリュームやJupyterLabワークスペースのほぼ瞬時のプロビジョニング、データボリュームやJupyterLabワークスペースのほぼ瞬時のクローニングなど、さまざまなデータ管理タスクを簡単に実行できます。 トレーサビリティやベースライン化のために、データボリュームまたはJupyterLabワークスペースのスナップショットをほぼ瞬時に作成できます。このPythonライブラリは、任意

のPythonプログラムまたはJupyterノートブックに読み込むことができるコマンドラインユーティリティまたは関数のライブラリとして機能します。

NVIDIA Triton 推論サーバ

NVIDIA Triton Inference Serverは、オープンソースの推論サービスソフトウェアです。モデルの導入と実行を標準化し、高速で拡張性に優れたAIを本番環境に提供できます。Triton Inference Serverは、GPUベースまたはCPUベースのインフラ上のあらゆるフレームワークからトレーニング済みAIモデルを導入、実行、拡張できるため、AI推論が合理化されます。Triton Inference Serverは、TensorFlow、NVIDIA TensorRT、PyTorch、MXNetなどの主要なフレームワークをすべてサポートします。 OpenVNOなど。TritonはKubernetesと統合し、オーケストレーションと拡張を実現します。主要なパブリッククラウドのAIプラットフォームとKubernetesプラットフォームで使用できます。また、多くのMLOpsソフトウェアソリューションと統合されています。

PyTorch

"PyTorch" はオープンソースのMLフレームワークです。GPUとCPUを使用するディープラーニング用に最適化されたテンソルライブラリです。PyTorchパッケージには多次元テンソル用のデータ構造が含まれており、他の有用なユーティリティ間でテンソルを効率的にシリアル化するための多くのユーティリティを提供します。また、コンピューティング機能を備えたNVIDIA GPUでテンソル計算を実行できるCUDA対応製品もあります。この検証では、OpenCV-Python (CV2)ライブラリを使用してモデルを検証しながら、Pythonで最も直感的なコンピュータビジョンの概念を活用しています。

データ管理を簡易化

データ管理は、AIアプリケーションの運用やAI / MLデータセットのトレーニングに適切なリソースを使用できるように、エンタープライズIT運用とデータサイエンティストにとって非常に重要です。以下に記載するネットアップテクノロジに関する追加情報 は、この検証の対象外ですが、導入環境によっては関連性がある場合もあります。

ONTAP データ管理ソフトウェアには、運用を合理化および簡易化し、総運用コストを削減するための次の機能が含まれています。

- インラインデータコンパクション、強化された重複排除：データコンパクションはストレージブロック内の無駄なスペースを削減し、重複排除は実効容量を大幅に増やします。この環境データはローカルに格納され、データはクラウドに階層化されます。
- 最小、最大、アダプティブのQuality of Service（AQoS）。きめ細かいサービス品質（QoS）管理機能により、高度に共有された環境で重要なアプリケーションのパフォーマンスレベルを維持できます。
- NetApp FabricPool の略。Amazon Web Services（AWS）、Azure、NetApp StorageGRID ストレージ解決策 など、パブリッククラウドとプライベートクラウドのストレージオプションへコールドデータを自動的に階層化します。FabricPool の詳細については、を参照してください "TR-4598：『FabricPool best bests』"。

データの高速化と保護

ONTAP は、卓越したパフォーマンスとデータ保護を実現し、以下の方法でこれらの機能を拡張します。

- パフォーマンスとレイテンシの低下：ONTAP は、可能なかぎり最小のレイテンシで最高のスループットを提供します。
- データ保護ONTAP には、組み込みのデータ保護機能が用意されており、すべてのプラットフォームを共通の管理機能で管理できます。

- NetApp Volume Encryption（NVE）：ONTAP は、オンボードと外部キー管理の両方をサポートし、ボリュームレベルでのネイティブな暗号化を実現します。
- マルチテナンシーおよび多要素認証ONTAP を使用すると、最高レベルのセキュリティでインフラリソースを共有できます。

将来のニーズにも対応できるインフラ

ONTAP は、次の機能を備えており、要件が厳しく、絶えず変化するビジネスニーズに対応できます。

- シームレスな拡張とノンストップオペレーションONTAP を使用すると、既存のコントローラとスケールアウトクラスタに無停止で容量を追加できます。NVMe や 32Gb FC などの最新テクノロジへのアップグレードも、コストのかかるデータ移行やシステム停止を行わずに実行できます。
- クラウドへの接続：ONTAP は、すべてのパブリッククラウドでSoftware-Defined Storage（ONTAP Select ）とクラウドネイティブインスタンス（NetApp Cloud Volumes Service ）のオプションを選択できる、マルチクラウドに対応した最もクラウド対応のストレージ管理ソフトウェアです。
- 新しいアプリケーションとの統合：ONTAP は、既存のエンタープライズアプリケーションをサポートするインフラを使用して、自律走行車、スマートシティ、インダストリー4.0などの次世代プラットフォームやアプリケーション向けにエンタープライズクラスのデータサービスを提供します。

ネットアップアストラコントロール

ネットアップの Astra 製品ファミリーは、オンプレミスとパブリッククラウドの Kubernetes アプリケーション向けに、ネットアップのストレージテクノロジとデータ管理テクノロジを基盤とするストレージサービスとアプリケーション対応データ管理サービスを提供します。Kubernetesアプリケーションのバックアップ、データの別のクラスタへの移行、作業用アプリケーションのクローンの瞬時作成を簡単に実行できます。パブリッククラウドで実行されているKubernetesアプリケーションを管理する必要がある場合は、のドキュメントを参照してください "Astra 制御サービス"。Astra Control Service は、 Google Kubernetes Engine （ GKE ）および Azure Kubernetes Service （ AKS ）で Kubernetes クラスタのアプリケーション対応データ管理を提供する、ネットアップが管理するサービスです。

ネットアップアストラト **Trident**

アストラ "Trident" ネットアップは、 Docker と Kubernetes 向けのオープンソースの動的ストレージオーケストレーションツールであり、永続的ストレージの作成、管理、使用を簡易化します。Kubernetesネイティブアプリケーションである Tridentは、Kubernetesクラスタ内で直接実行されます。Trident を使用すると、 DL コンテナイメージをネットアップストレージにシームレスに導入し、エンタープライズクラスの AI コンテナ環境を実現できます。Kubernetesユーザ（ML開発者、データサイエンティストなど）は、オーケストレーションとクローニングを作成、管理、自動化し、ネットアップテクノロジを基盤とする高度なデータ管理機能を活用できます。

NetApp Cloud Sync の略

"Cloud Sync" 迅速かつセキュアなデータ同期を実現するネットアップのサービスです。オンプレミスの NFS または SMB ファイル共有、 NetApp StorageGRID 、 NetApp ONTAP S3 、 NetApp Cloud Volumes Service 、 Azure NetApp Files 、 Amazon Simple Storage Service （ Amazon S3 ）、 Amazon Elastic File System （ Amazon EFS ）、 Azure Blob 、 Google Cloud Storage 間でファイルを転送する必要があるかどうか または、 IBM Cloud Object Storage を使用すると、 Cloud Sync で必要な場所に迅速かつ安全にファイルを移動できます。転送されたデータは、ソースとターゲットの両方で完全に使用できます。Cloud Sync は、事前に定義されたスケジュールに基づいてデータを継続的に同期し、差分のみを移動するため、データレプリケーションにかかる時間とコストを最小限に抑えることができます。Cloud Sync は、セットアップや使用がきわめて簡単なソフトウェアサービス（SaaS）ツールです。Cloud Sync によって実行されるデータ転送は、データブ

ローカーによって実行されます。Cloud Sync データブローカーは、 AWS 、 Azure 、 Google Cloud Platform 、オンプレミスに導入できます。

ネットアップのクラウドデータの意味

強力なAIアルゴリズム、 "ネットアップのクラウドデータの意味" データ資産全体の管理とデータガバナンスを自動化します。コスト削減を容易に特定し、コンプライアンスやプライバシーに関する懸念を特定し、最適化の機会を見つけることができます。クラウドデータセンスダッシュボードを使用すると、重複データを特定して冗長性を排除し、個人データ、個人データ、機密データをマッピングし、機密データや異常に関するアラートを有効にすることができます。

"次のステップ：テストと検証の計画"

テストと検証の計画

"前のページ：テクノロジの概要"

この解決策 設計では、次の3つのシナリオが検証されました。

- Kubernetes向けNetApp DataOpsツールキットを使用してオーケストレーションされたJupyterLabワークスペース内で、Protopia難読化を使用するかどうかに関係なく、推論タスクを実行します。
- Kubernetesでの一括推論ジョブ。Protopia難読化を使用するかどうかにかかわらず、NetApp DataOps Toolkit for Kubernetesを使用してオーケストレーションされたデータボリュームを使用します。
- Kubernetes向けNetApp DataOpsツールキットを使用してオーケストレーションされた、NVIDIA Triton Inference Serverインスタンスを使用した推論タスク。Triton推論APIを呼び出す前に、Protopia難読化を画像に適用して、ネットワーク経由で送信されるデータを難読化する必要がある一般的な要件をシミュレートしました。このワークフローは、信頼できるゾーン内でデータが収集され、推論のためにその信頼できるゾーンの外部に渡される必要があるユースケースに該当します。Protopia難読化を使用しないと'機密データが信頼ゾーンから離れることなく'このタイプのワークフローを実装できません

"次の手順：設定をテストします。"

設定をテストします

"前：テストと検証の計画"

次の表に、解決策 設計検証環境の概要を示します。

コンポーネント	バージョン
Kubernetes	1.1.6
NetApp Astra Trident CSIドライバ	22.01.0
Kubernetes向けNetApp DataOpsツールキット	2.3.0
NVIDIA Triton 推論サーバ	21.11-py3.

"次の手順：手順をテストします。"

手順をテストします

"前の手順：設定をテストします。"

このセクションでは、検証を完了するために必要なタスクについて説明します。

前提条件

このセクションで説明するタスクを実行するには、次のツールをインストールして設定したLinuxまたはmacOSホストにアクセスできる必要があります。

- Kubectl（既存のKubernetesクラスタへのアクセスを設定）
  - インストールと設定の手順については、を参照してください "こちらをご覧ください"。
- Kubernetes向けNetApp DataOpsツールキット
  - インストール手順が記載されています "こちらをご覧ください"。

シナリオ**1–JupyterLab**におけるオンデマンド推論

1. AI / ML推論ワークロード用のKubernetesネームスペースを作成します。

```
$ kubectl create namespace inference
namespace/inference created
```

2. NetApp DataOpsツールキットを使用して、推論を実行するデータを格納する永続的ボリュームをプロビジョニングします。

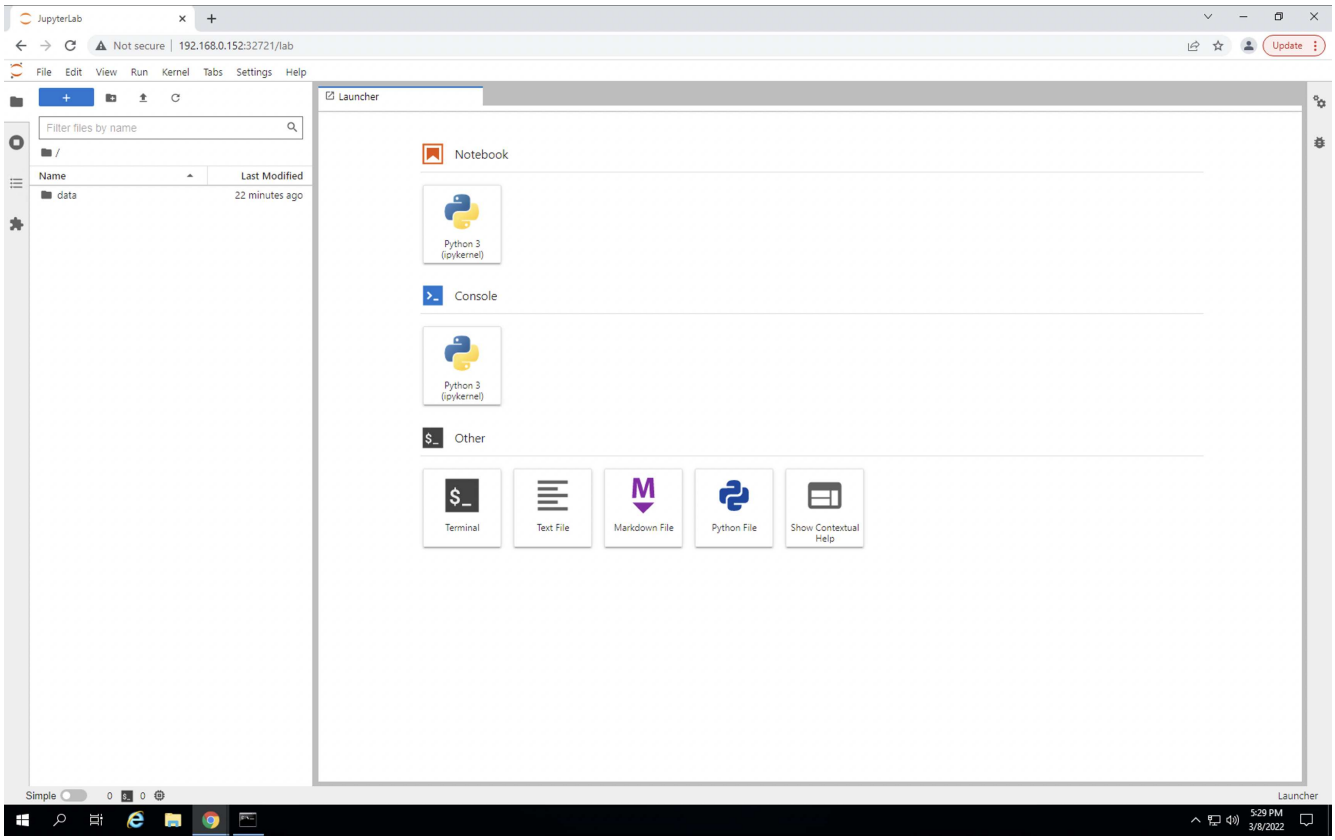
```
$ netapp_dataops_k8s_cli.py create volume --namespace=inference --pvc
-name=inference-data --size=50Gi
Creating PersistentVolumeClaim (PVC) 'inference-data' in namespace
'inference'.
PersistentVolumeClaim (PVC) 'inference-data' created. Waiting for
Kubernetes to bind volume to PVC.
Volume successfully created and bound to PersistentVolumeClaim (PVC)
'inference-data' in namespace 'inference'.
```

3. NetApp DataOpsツールキットを使用して、JupyterLabの新しいワークスペースを作成します。前の手順で作成した永続ボリュームを'--mount-pvcオプションを使用してマウントします必要に応じて'--nvidia -GPU'オプションを使用して'NVIDIA GPUをワークスペースに割り当てます

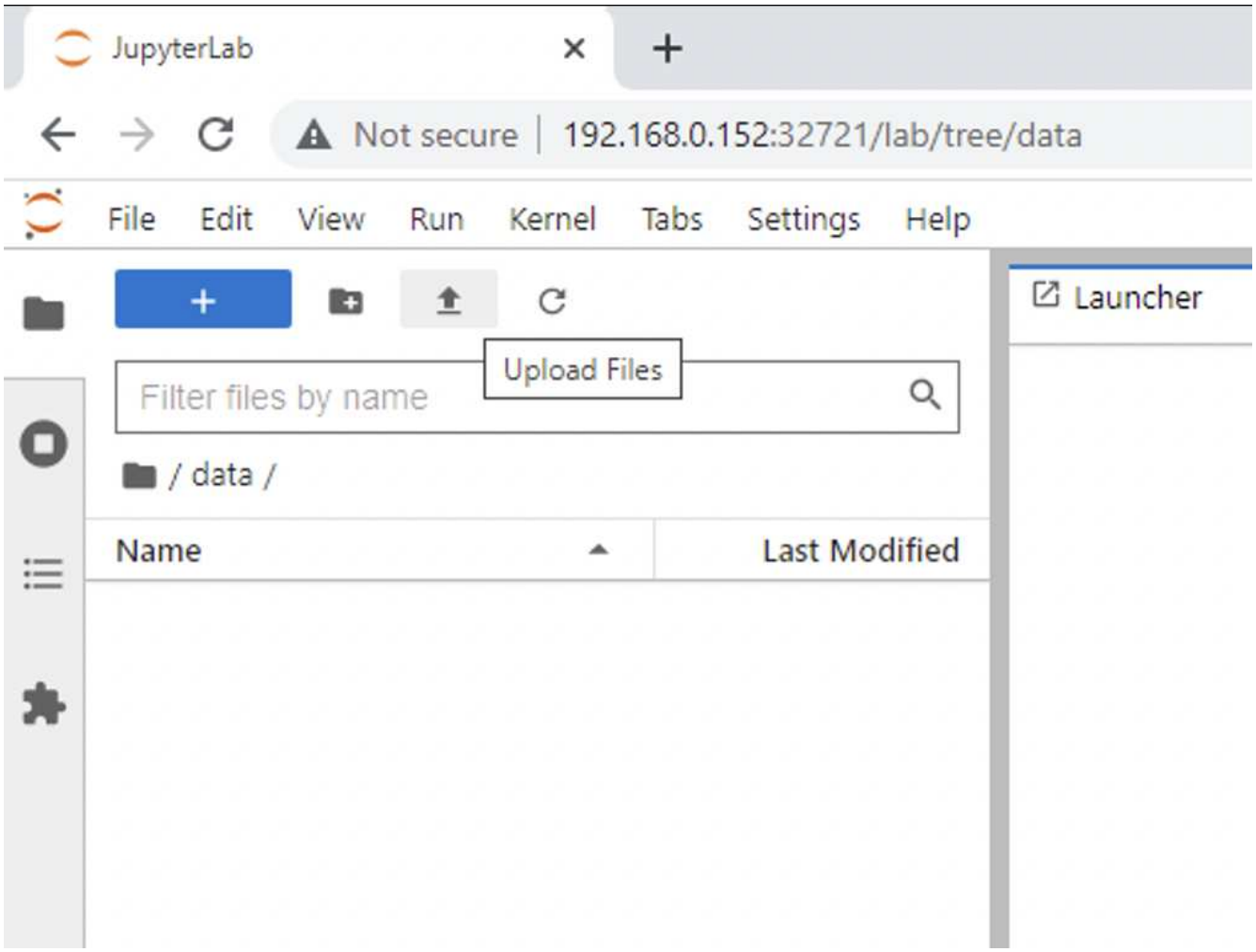
   次の例では'永続ボリュームinerial-data'が'/home/jovyan/data'のJupyterLabワークスペースコンテナにマウントされています公式のProject Jupyterコンテナイメージを使用する場合、「/home/jovyan」はJupyterLab Webインターフェイス内の最上位ディレクトリとして表示されます。

```
$ netapp_dataops_k8s_cli.py create jupyterlab --namespace=inference
--workspace-name=live-inference --size=50Gi --nvidia-gpu=2 --mount
-pvc=inference-data:/home/jovyan/data
Set workspace password (this password will be required in order to
access the workspace):
Re-enter password:
Creating persistent volume for workspace...
Creating PersistentVolumeClaim (PVC) 'ntap-dsutil-jupyterlab-live-
inference' in namespace 'inference'.
PersistentVolumeClaim (PVC) 'ntap-dsutil-jupyterlab-live-inference'
created. Waiting for Kubernetes to bind volume to PVC.
Volume successfully created and bound to PersistentVolumeClaim (PVC)
'ntap-dsutil-jupyterlab-live-inference' in namespace 'inference'.
Creating Service 'ntap-dsutil-jupyterlab-live-inference' in namespace
'inference'.
Service successfully created.
Attaching Additional PVC: 'inference-data' at mount_path:
'/home/jovyan/data'.
Creating Deployment 'ntap-dsutil-jupyterlab-live-inference' in namespace
'inference'.
Deployment 'ntap-dsutil-jupyterlab-live-inference' created.
Waiting for Deployment 'ntap-dsutil-jupyterlab-live-inference' to reach
Ready state.
Deployment successfully created.
Workspace successfully created.
To access workspace, navigate to http://192.168.0.152:32721
```

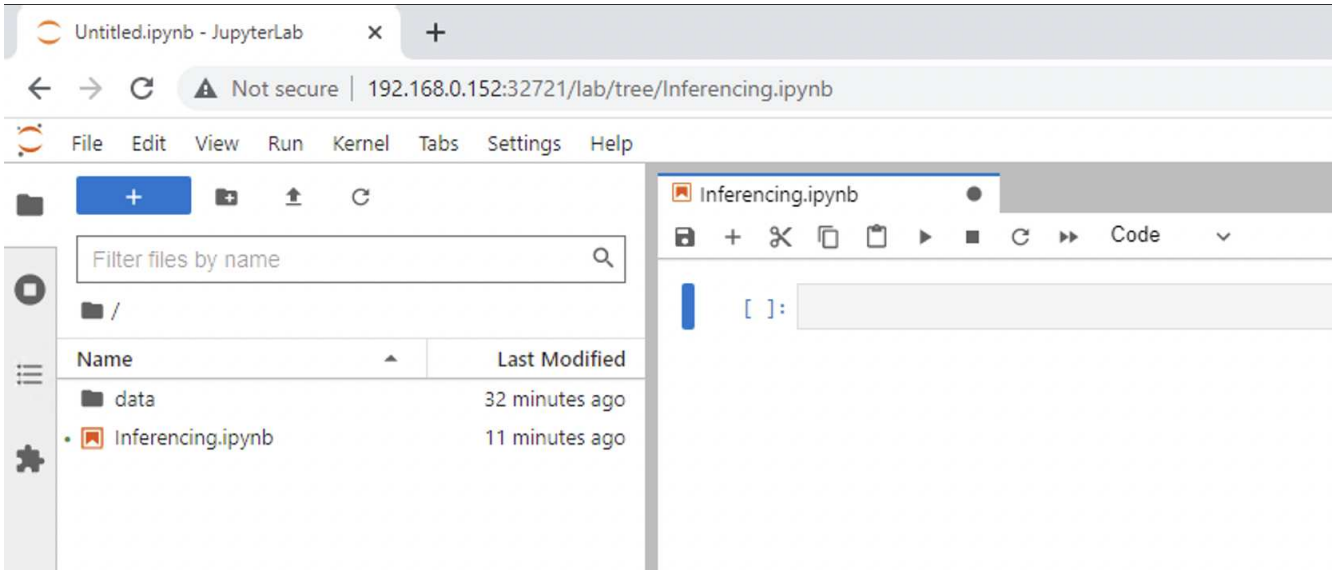
4. 「create jupyterlab」コマンドの出力で指定したURLを使用して、JupyterLabワークスペースにアクセスします。データディレクトリは、ワークスペースにマウントされた永続ボリュームを表します。



5. 「data」ディレクトリを開き、推論を実行するファイルをアップロードします。ファイルがデータディレクトリにアップロードされると、ワークスペースにマウントされた永続ボリュームに自動的に保存されます。ファイルをアップロードするには、次の図に示すように、[ファイルのアップロード]アイコンをクリックします。



6. トップレベルのディレクトリに戻り、新しいノートブックを作成します。



7. ノートブックに推論コードを追加します。次の例は、イメージ検出のユースケースの推論コードを示しています。

Launcher | image-demo-pytorch.ipynb

Code | Python 3 (ipykernel)

STEP 3-1: Clean (Without obfuscation) detection

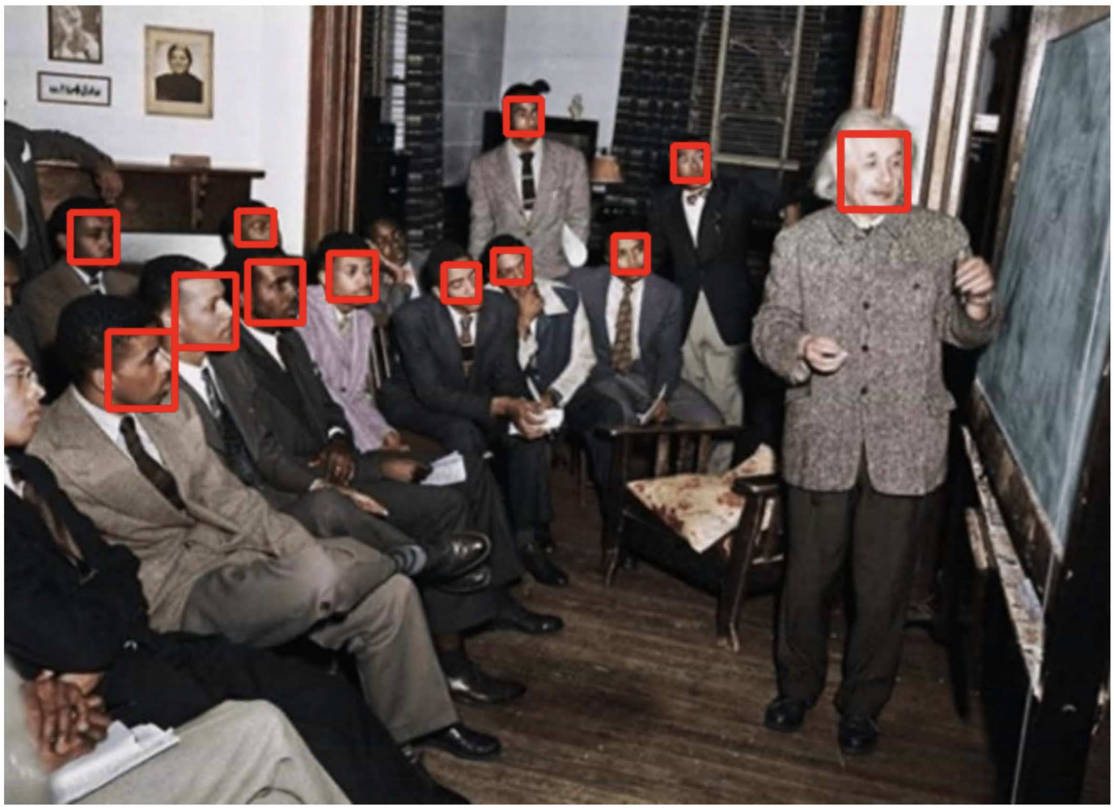
[9]:
```
# get current frame
frame = input_image

# preprocess input
preprocessed_input = preprocess_input(frame)
preprocessed_input = torch.Tensor(preprocessed_input).to(device)

# run forward pass
clean_activation = clean_model.forward_head(preprocessed_input)  # runs the first few layers
loc, pred = clean_model.forward_tail(clean_activation)  # runs rest of the layers

# postprocess output
clean_pred = (loc.detach().cpu().numpy(), pred.detach().cpu().numpy())
clean_outputs = postprocess_outputs(
    clean_pred, [[input_image_width, input_image_height]], priors, THRESHOLD
)

# draw rectangles
clean_frame = copy.deepcopy(frame)  # needs to be deep copy
for (x1, y1, x2, y2, s) in clean_outputs[0]:
    x1, y1 = int(x1), int(y1)
    x2, y2 = int(x2), int(y2)
    cv2.rectangle(clean_frame, (x1, y1), (x2, y2), (0, 0, 255), 4)
```

Launcher | image-demo-pytorch.ipynb

Code | Python 3 (ipykernel)

Visualize Clean (Without obfuscation) detection

[10]:
```
show_cv2_image(clean_frame, scale=2)
```



8. 推測コードにProtopia難読化を追加します。Protopiaは、お客様と直接協力してユースケースに固有のドキュメントを提供しますが、本書では取り上げません。次の例は、Protopia難読化を追加した場合のイメージ検出の推論コードを示しています。

Launcher | image-demo-pytorch.ipynb

Code | Python 3 (ipykernel)

STEP 3-2: Protopia AI (With obfuscation) detection

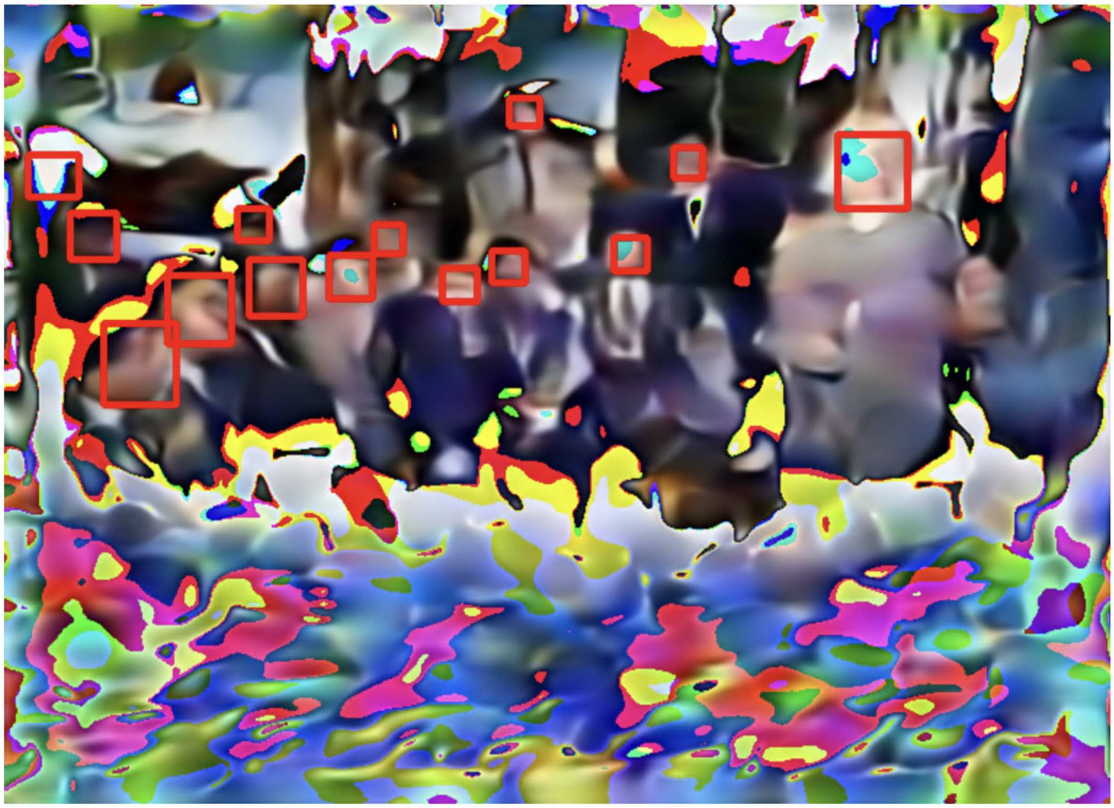
[11]:
```
# get current frame
frame = input_image

# preprocess input
preprocessed_input = preprocess_input(frame)
preprocessed_input = torch.Tensor(preprocessed_input).to(device)

# run forward pass
not_noisy_activation = noisy_model.forward_head(preprocessed_input)  # runs the first few layers
#################################################################
#          SINGLE ADITIONAL LINE FOR PRIVATE INFERENCE          #
#################################################################
noisy_activation = noisy_model.forward_noise(not_noisy_activation)
#################################################################
loc, pred = noisy_model.forward_tail(noisy_activation)  # runs rest of the layers

# postprocess output
noisy_pred = (loc.detach().cpu().numpy(), pred.detach().cpu().numpy())
noisy_outputs = postprocess_outputs(
    noisy_pred, [[input_image_width, input_image_height]], priors, THRESHOLD * 0.5
)

# get reconstruction of the noisy activation
noisy_reconstruction = decoder_function(noisy_activation)
noisy_reconstruction = noisy_reconstruction.detach().cpu().numpy()[0]
noisy_reconstruction = unpreprocess_output(
    noisy_reconstruction, (input_image_width, input_image_height), True
).astype(np.uint8)

# draw rectangles
for (x1, y1, x2, y2, s) in noisy_outputs[0]:
    x1, y1 = int(x1), int(y1)
    x2, y2 = int(x2), int(y2)
    cv2.rectangle(noisy_reconstruction, (x1, y1), (x2, y2), (0, 0, 255), 4)
```

Launcher | image-demo-pytorch.ipynb

Code | Python 3 (ipykernel)

Visualize Protopia AI (With obfuscation) detection

[12]:
```
show_cv2_image(noisy_reconstruction, scale=2)
```



シナリオ2–Kubernetesでのバッチ推論

1. AI / ML推論ワークロード用のKubernetesネームスペースを作成します。

```
$ kubectl create namespace inference
namespace/inference created
```

2. NetApp DataOpsツールキットを使用して、推論を実行するデータを格納する永続的ボリュームをプロビジョニングします。

```
$ netapp_dataops_k8s_cli.py create volume --namespace=inference --pvc
-name=inference-data --size=50Gi
Creating PersistentVolumeClaim (PVC) 'inference-data' in namespace
'inference'.
PersistentVolumeClaim (PVC) 'inference-data' created. Waiting for
Kubernetes to bind volume to PVC.
Volume successfully created and bound to PersistentVolumeClaim (PVC)
'inference-data' in namespace 'inference'.
```

3. 新しい永続ボリュームに、推論を実行するデータを入力します。

   PVCにデータをロードする方法はいくつかあります。データがNetApp StorageGRID やAmazon S3などのS3互換オブジェクトストレージプラットフォームに現在格納されている場合は、を使用できます"NetApp DataOpsツールキットS3 Data Moverの機能"。また、JupyterLabワークスペースを作成し、JupyterLab Webインターフェイスを使用してファイルをアップロードする方法も簡単です。手順3〜5を参照してください[Scenario 1 – On-demand inferencing in JupyterLab]」

4. バッチ推論タスク用のKubernetesジョブを作成します。次の例は、イメージ検出のユースケースに対するバッチ推論ジョブを示しています。このジョブは、一連のイメージ内の各イメージに対して推論を実行し、stdoutに推論の精度の指標を書き込みます。

```
$ vi inference-job-raw.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-inference-raw
  namespace: inference
spec:
  backoffLimit: 5
  template:
    spec:
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: inference-data
      - name: dshm
        emptyDir:
          medium: Memory
      containers:
      - name: inference
        image: netapp-protopia-inference:latest
        imagePullPolicy: IfNotPresent
        command: ["python3", "run-accuracy-measurement.py", "--dataset",
"/data/netapp-face-detection/FDDB"]
        resources:
          limits:
            nvidia.com/gpu: 2
        volumeMounts:
        - mountPath: /data
          name: data
        - mountPath: /dev/shm
          name: dshm
      restartPolicy: Never
$ kubectl create -f inference-job-raw.yaml
job.batch/netapp-inference-raw created
```

5. 推論ジョブが正常に完了したことを確認します。

```
$ kubectl -n inference logs netapp-inference-raw-255sp
100%|██████████| 89/89 [00:52<00:00,  1.68it/s]
Reading Predictions : 100%|██████████| 10/10 [00:01<00:00,  6.23it/s]
Predicting ... : 100%|██████████| 10/10 [00:16<00:00,  1.64s/it]
==================== Results ====================
FDDB-fold-1 Val AP: 0.9491256561145955
FDDB-fold-2 Val AP: 0.9205024466101926
FDDB-fold-3 Val AP: 0.9253013871078468
FDDB-fold-4 Val AP: 0.9399781485863011
FDDB-fold-5 Val AP: 0.9504280149478732
FDDB-fold-6 Val AP: 0.9416473519339292
FDDB-fold-7 Val AP: 0.9241631566241117
FDDB-fold-8 Val AP: 0.9072663297546659
FDDB-fold-9 Val AP: 0.9339648715035469
FDDB-fold-10 Val AP: 0.9447707905560152
FDDB Dataset Average AP: 0.9337148153739079
==================================================
mAP: 0.9337148153739079
```

6. 推測ジョブにProtopia難読化を追加します。Protopiaの難読化を追加する手順は、このテクニカルレポートでは説明していませんが、Protopiaから直接追加できます。次の例は、アルファ値0.8を使用してProtopia難読化を行った場合のフェース検出のバッチ推論ジョブを示しています。このジョブは、一連のイメージ内の各イメージに対して推論を実行する前にProtopia難読化を適用し、stdoutに推論の精度指標を書き込みます。

   このステップは、アルファ値0.05、0.1、0.2、0.4、0.6について繰り返しました。 0.8、0.9、および0.95。結果はに表示されます "「推論の精度比較」"

```
$ vi inference-job-protopia-0.8.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-inference-protopia-0.8
  namespace: inference
spec:
  backoffLimit: 5
  template:
    spec:
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: inference-data
      - name: dshm
        emptyDir:
          medium: Memory
      containers:
      - name: inference
        image: netapp-protopia-inference:latest
        imagePullPolicy: IfNotPresent
        env:
        - name: ALPHA
          value: "0.8"
        command: ["python3", "run-accuracy-measurement.py", "--dataset",
"/data/netapp-face-detection/FDDB", "--alpha", "$(ALPHA)", "--noisy"]
        resources:
          limits:
            nvidia.com/gpu: 2
        volumeMounts:
        - mountPath: /data
          name: data
        - mountPath: /dev/shm
          name: dshm
      restartPolicy: Never
$ kubectl create -f inference-job-protopia-0.8.yaml
job.batch/netapp-inference-protopia-0.8 created
```

7. 推論ジョブが正常に完了したことを確認します。

```
$ kubectl -n inference logs netapp-inference-protopia-0.8-b4dkz
100%|██████████| 89/89 [01:05<00:00,  1.37it/s]
Reading Predictions : 100%|██████████| 10/10 [00:02<00:00,  3.67it/s]
Predicting ... : 100%|██████████| 10/10 [00:22<00:00,  2.24s/it]
==================== Results ====================
FDDB-fold-1 Val AP: 0.8953066115834589
FDDB-fold-2 Val AP: 0.8819580264029936
FDDB-fold-3 Val AP: 0.8781107458462862
FDDB-fold-4 Val AP: 0.9085731346308461
FDDB-fold-5 Val AP: 0.9166445508275378
FDDB-fold-6 Val AP: 0.9101178994188819
FDDB-fold-7 Val AP: 0.8383443678423771
FDDB-fold-8 Val AP: 0.8476311547659464
FDDB-fold-9 Val AP: 0.8739624502111121
FDDB-fold-10 Val AP: 0.8905468076424851
FDDB Dataset Average AP: 0.8841195749171925
==================================================
mAP: 0.8841195749171925
```

シナリオ3–NVIDIA Triton Inference Server

1. AI / ML推論ワークロード用のKubernetesネームスペースを作成します。

```
$ kubectl create namespace inference
namespace/inference created
```

2. NetApp DataOpsツールキットを使用して、NVIDIA Triton Inference Serverのモデルリポジトリとして使用する永続的ボリュームをプロビジョニングします。

```
$ netapp_dataops_k8s_cli.py create volume --namespace=inference --pvc
-name=triton-model-repo --size=100Gi
Creating PersistentVolumeClaim (PVC) 'triton-model-repo' in namespace
'inference'.
PersistentVolumeClaim (PVC) 'triton-model-repo' created. Waiting for
Kubernetes to bind volume to PVC.
Volume successfully created and bound to PersistentVolumeClaim (PVC)
'triton-model-repo' in namespace 'inference'.
```

3. の新しい永続ボリュームにモデルを保存します "の形式で入力し" これはNVIDIA Triton Inference Serverによって認識されます。

   PVCにデータをロードする方法はいくつかあります。簡単な方法としては、「」の手順3〜5で説明しているように、JupyterLabワークスペースを作成し、JupyterLab Webインターフェイスを使用してファイルを

アップロードする方法があります[Scenario 1 – On-demand inferencing in JupyterLab]。」

4. NetApp DataOpsツールキットを使用して、新しいNVIDIA Triton Inference Serverインスタンスを導入します。

```
$ netapp_dataops_k8s_cli.py create triton-server --namespace=inference
--server-name=netapp-inference --model-repo-pvc-name=triton-model-repo
Creating Service 'ntap-dsutil-triton-netapp-inference' in namespace
'inference'.
Service successfully created.
Creating Deployment 'ntap-dsutil-triton-netapp-inference' in namespace
'inference'.
Deployment 'ntap-dsutil-triton-netapp-inference' created.
Waiting for Deployment 'ntap-dsutil-triton-netapp-inference' to reach
Ready state.
Deployment successfully created.
Server successfully created.
Server endpoints:
http: 192.168.0.152: 31208
grpc: 192.168.0.152: 32736
metrics: 192.168.0.152: 30009/metrics
```

5. 推論タスクを実行するには、TritonクライアントSDKを使用します。次のPythonコードの抜粋では、Triton PythonクライアントSDKを使用して、フェース検出のユースケースに対する推論タスクを実行しています。この例では、推論のためにTriton APIを呼び出し、イメージを渡します。次に、Triton Inference Serverが要求を受信し、モデルを呼び出して、API結果の一部として推論出力を返します。

```
# get current frame
frame = input_image
# preprocess input
preprocessed_input = preprocess_input(frame)
preprocessed_input = torch.Tensor(preprocessed_input).to(device)
# run forward pass
clean_activation = clean_model_head(preprocessed_input)  # runs the
first few layers
#####################################################################
##############
#          pass clean image to Triton Inference Server API for
inferencing           #
#####################################################################
##############
triton_client =
httpclient.InferenceServerClient(url="192.168.0.152:31208",
verbose=False)
model_name = "face_detection_base"
inputs = []
```

```
outputs = []
inputs.append(httpclient.InferInput("INPUT__0", [1, 128, 32, 32],
"FP32"))
inputs[0].set_data_from_numpy(clean_activation.detach().cpu().numpy(),
binary_data=False)
outputs.append(httpclient.InferRequestedOutput("OUTPUT__0",
binary_data=False))
outputs.append(httpclient.InferRequestedOutput("OUTPUT__1",
binary_data=False))
results = triton_client.infer(
    model_name,
    inputs,
    outputs=outputs,
    #query_params=query_params,
    headers=None,
    request_compression_algorithm=None,
    response_compression_algorithm=None)
#print(results.get_response())
statistics =
triton_client.get_inference_statistics(model_name=model_name,
headers=None)
print(statistics)
if len(statistics["model_stats"]) != 1:
    print("FAILED: Inference Statistics")
    sys.exit(1)

loc_numpy = results.as_numpy("OUTPUT__0")
pred_numpy = results.as_numpy("OUTPUT__1")
#######################################################################
##############
# postprocess output
clean_pred = (loc_numpy, pred_numpy)
clean_outputs = postprocess_outputs(
    clean_pred, [[input_image_width, input_image_height]], priors,
THRESHOLD
)
# draw rectangles
clean_frame = copy.deepcopy(frame)  # needs to be deep copy
for (x1, y1, x2, y2, s) in clean_outputs[0]:
    x1, y1 = int(x1), int(y1)
    x2, y2 = int(x2), int(y2)
    cv2.rectangle(clean_frame, (x1, y1), (x2, y2), (0, 0, 255), 4)
```

6. 推測コードにProtopia難読化を追加します。Protopia難読化を追加する手順はProtopiaから直接確認できますが、この手順については本テクニカルレポートでは説明していません。次の例は、前述の手順5と同じPythonコードを示していますが、Protopia難読化が追加されています。

Triton APIに渡される前に、Protopia難読化が画像に適用されることに注意してください。このため、難読化されていない画像はローカルマシンから離れることはありません。難読化されたイメージだけがネットワークを通過します。このワークフローは、信頼できるゾーン内でデータが収集され、推論のためにその信頼できるゾーンの外部に渡す必要があるユースケースに該当します。Protopiaの難読化がなければ、機密データが信頼できるゾーンから離れることなく、このタイプのワークフローを実装することはできません。

```
# get current frame
frame = input_image
# preprocess input
preprocessed_input = preprocess_input(frame)
preprocessed_input = torch.Tensor(preprocessed_input).to(device)
# run forward pass
not_noisy_activation = noisy_model_head(preprocessed_input)  # runs the
first few layers
#################################################################
#          obfuscate image locally prior to inferencing          #
#          SINGLE ADITIONAL LINE FOR PRIVATE INFERENCE           #
#################################################################
noisy_activation = noisy_model_noise(not_noisy_activation)
#################################################################
#######################################################################
####################
#          pass obfuscated image to Triton Inference Server API for
inferencing           #
#######################################################################
####################
triton_client =
httpclient.InferenceServerClient(url="192.168.0.152:31208",
verbose=False)
model_name = "face_detection_noisy"
inputs = []
outputs = []
inputs.append(httpclient.InferInput("INPUT__0", [1, 128, 32, 32],
"FP32"))
inputs[0].set_data_from_numpy(noisy_activation.detach().cpu().numpy(),
binary_data=False)
outputs.append(httpclient.InferRequestedOutput("OUTPUT__0",
binary_data=False))
outputs.append(httpclient.InferRequestedOutput("OUTPUT__1",
binary_data=False))
results = triton_client.infer(
    model_name,
    inputs,
    outputs=outputs,
    #query_params=query_params,
    headers=None,
```

```
    request_compression_algorithm=None,
    response_compression_algorithm=None)
#print(results.get_response())
statistics =
triton_client.get_inference_statistics(model_name=model_name,
headers=None)
print(statistics)
if len(statistics["model_stats"]) != 1:
    print("FAILED: Inference Statistics")
    sys.exit(1)

loc_numpy = results.as_numpy("OUTPUT__0")
pred_numpy = results.as_numpy("OUTPUT__1")
#####################################################################
###################

# postprocess output
noisy_pred = (loc_numpy, pred_numpy)
noisy_outputs = postprocess_outputs(
    noisy_pred, [[input_image_width, input_image_height]], priors,
THRESHOLD * 0.5
)
# get reconstruction of the noisy activation
noisy_reconstruction = decoder_function(noisy_activation)
noisy_reconstruction = noisy_reconstruction.detach().cpu().numpy()[0]
noisy_reconstruction = unpreprocess_output(
    noisy_reconstruction, (input_image_width, input_image_height), True
).astype(np.uint8)
# draw rectangles
for (x1, y1, x2, y2, s) in noisy_outputs[0]:
    x1, y1 = int(x1), int(y1)
    x2, y2 = int(x2), int(y2)
    cv2.rectangle(noisy_reconstruction, (x1, y1), (x2, y2), (0, 0, 255),
4)
```

"次の例は、推論の精度の比較です。"

推論の精度の比較

"前へ：手順のテスト。"

この検証では、rawイメージのセットを使用して、イメージ検出のユースケースに対して推論を実行しました。次に、同じイメージセットに対して、推論の前にProtopia難読化が追加された同じ推論タスクを実行しました。このタスクでは、Protopia難読化コンポーネントに異なる値のalphaを使用しています。Protopia難読化のコンテキストでは、アルファ値は適用される難読化の量を表し、アルファ値が大きいほど難読化のレベルが高くなります。次に、これらの異なる実行間の推論の精度を比較しました。

以下の2つの表に、ネットアップのユースケースとその概要を示します。

Protopiaは、お客様と直接協力して、特定のユースケースに適したアルファ値を決定します。

コンポーネント	詳細
モデル	フェースボックス(PyTorch)-
データセット	FDDBデータセット

Protopia難読化	アルファ	精度
いいえ	該当なし	0.9337148153739079
はい。	0.05	0.9028766627325002
はい。	0.1	0.9024301009661478
はい。	0.2	0.9081836283186224
はい。	0.4	0.9073066107482036
はい。	0.6	0.8847816568680239
はい。	0.8	0.8841195749171925
はい。	0.9	0.8455427675252052
はい。	0.95	0.8455427675252052

"次の例は、難読化の速度です。"

難読化の速度

"前の例：推論の精度の比較。"

この検証では、Protopia難読化を1920 x 1080ピクセル画像に5回適用し、難読化手順が完了するまでに毎回かかった時間を測定しました。単一のNVIDIA V100 GPUで動作するPyTorchを使用して難読化を適用し、実行間のGPUキャッシュをクリアしました。難読化手順では、5回の実行でそれぞれ5.47ミリ秒、5.27ミリ秒、4.54ミリ秒、5.24ミリ秒、および4.84ミリ秒が実行されました。平均速度は5.072msでした。

"次は終わりです"

まとめ

"前のバージョン：難読化の速度。"

データは、保管中、転送中、コンピューティング中の3つの状態に存在します。AI推論サービスの重要な部分は、プロセス全体における脅威からのデータの保護である必要があります。推論の実行中にデータを保護することが重要です。これは、外部のお客様と推論サービスを提供するビジネスの両方の個人情報がこのプロセスによって公開される可能性があるためです。Protopia AIは、今日の市場における機密性の高いAI推論を行うための、悪意のないソフトウェアのみの解決策 です。Protopiaを使用すると、AIには、AI / MLタスクを実行するために必要なデータレコードの変換された情報のみが渡されます。これ以外にも、データレコードには何も入力されません。この確率的変化はマスキングの形式ではなく、キュレーションされたノイズを使用してデータの表現を数学的に変更することに基づいています。

ONTAP 機能を備えたネットアップのストレージシステムは、ローカルSSDストレージと同等以上のパフォーマンスを提供します。また、NetApp DataOpsツールキットと組み合わせることで、データサイエンティスト、データエンジニア、AI / ML開発者、ビジネスまたはエンタープライズITの意思決定者に次のようなメリットをもたらします。

- AI システム、分析などの重要なビジネスシステム間でデータを容易に共有できます。このようなデータ共有により、インフラのオーバーヘッドを削減し、パフォーマンスを向上させ、企業全体のデータ管理を合理化できます。
- 個別に拡張可能なコンピューティングとストレージにより、コストを最小限に抑え、リソース使用率を向上させます。
- 統合された Snapshot コピーとクローンを使用して開発と導入のワークフローを合理化し、ユーザのワークスペースを瞬時にスペース効率よく利用できるほか、バージョン管理機能も統合され、導入も自動化されています。
- ディザスタリカバリ、ビジネス継続性、規制要件に対応するエンタープライズクラスのデータ保護とデータガバナンス。
- データ管理操作の簡単な呼び出し。データサイエンティストのワークスペースのSnapshotコピーを迅速に作成し、Jupyterノートブック内のNetApp DataOpsツールキットからバックアップとトレーサビリティを実現します。

ネットアップとProtopia解決策 は、エンタープライズクラスのAI推論環境に最適な、柔軟性に優れたスケールアウトアーキテクチャを提供します。データ保護を実現し、オンプレミス環境とハイブリッドクラウド環境の両方で、AI推論の機密要件を満たすことができる機密情報のプライバシーを提供します。

"Next：追加情報 、確認応答、およびバージョン履歴の参照先。"

追加情報 、確認応答、およびバージョン履歴の参照先

"前へ：終わりに。"

このドキュメントに記載されている情報の詳細については、以下のドキュメントや Web サイトを参照してください。

- NetApp ONTAP データ管理ソフトウェア—ONTAP 情報ライブラリ

  http://mysupport.netapp.com/documentation/productlibrary/index.html?productID=62286

- コンテナ向け NetApp 永続的ストレージ— NetApp Trident

  "https://netapp.io/persistent-storage-provisioner-for-kubernetes/"

- NetApp DataOps ツールキット

  "https://github.com/NetApp/netapp-dataops-toolkit"

- コンテナ向けNetApp Persistent Storage—NetApp Astra Trident

  "https://netapp.io/persistent-storage-provisioner-for-kubernetes/"

- Protopia AI—Confidential Inference（機密情報推論）

  "https://protopia.ai/blog/protopia-ai-takes-on-the-missing-link-in-ai-privacy-confidential-inference/"

- NetApp Cloud Sync の略

  "https://docs.netapp.com/us-en/occm/concept_cloud_sync.html#how-cloud-sync-works"

- NVIDIA Triton 推論サーバ

  "https://developer.nvidia.com/nvidia-triton-inference-server"

- NVIDIA Triton Inference Serverのドキュメント

  "https://docs.nvidia.com/deeplearning/triton-inference-server/index.html"

- PyTorchのフェイスボックス

  "https://github.com/zisianw/FaceBoxes.PyTorch"

謝辞

- ネットアップ、プリンシパルプロダクトマネージャー、Mark Cates氏
- ネットアップ、テクニカルマーケティングエンジニア、Sufian Ahmad氏
- 最高技術責任者兼Protopia AI教授、Hadi Esmaeilzadeh氏

バージョン履歴

バージョン	日付	ドキュメントバージョン履歴
バージョン 1.0 以降	2022年5月	初版リリース

著作権情報

Copyrightゥ2022 NetApp、Inc. All rights reserved.米国で印刷されていますこのドキュメントは著作権によって保護されています。画像媒体、電子媒体、および写真複写、記録媒体などの機械媒体など、いかなる形式および方法による複製も禁止します。 テープ媒体、または電子検索システムへの保管-著作権所有者の書面による事前承諾なし。

ネットアップの著作物から派生したソフトウェアは、次に示す使用許諾条項および免責条項の対象となります。

このソフトウェアは、ネットアップによって「現状のまま」提供されています。ネットアップは明示的、または商品性および特定目的に対する適合性の暗示的保証を含み、かつこれに限定されないいかなる暗示的な保証も行いません。ネットアップは、いかなる場合でも、間接的、偶発的、特別、懲罰的、または または結果的損害（代替品または代替サービスの調達、使用の損失、データ、利益、またはこれらに限定されないものを含みますが、これらに限定されません。） ただし、契約、厳格責任、または本ソフトウェアの使用に起因する不法行為（過失やその他を含む）のいずれであっても、かかる損害の可能性について知らされていた場合でも、責任の理論に基づいて発生します。

ネットアップは、ここに記載されているすべての製品に対する変更を随時、予告なく行う権利を保有します。ネットアップによる明示的な書面による合意がある場合を除き、ここに記載されている製品の使用により生じる責任および義務に対して、ネットアップは責任を負いません。この製品の使用または購入は、ネットアップの特許権、商標権、またはその他の知的所有権に基づくライセンスの供与とはみなされません。

このマニュアルに記載されている製品は、 1 つ以上の米国特許、その他の国の特許、および出願中の特許によ特許、その他の国の特許、および出願中の特許。

権利の制限について：政府による使用、複製、開示は、 DFARS 252.227-7103 （ 1988 年 10 月）および FAR 52-227-19 （ 1987 年 6 月）の Rights in Technical Data and Computer Software （技術データおよびコンピュータソフトウェアに関する諸権利）条項の（ c ）（ 1 ）（ ii ）項、に規定された制限が適用されます。

商標情報

NetApp、NetAppのロゴ、に記載されているマーク http://www.netapp.com/TM は、NetApp、Inc.の商標です。その他の会社名と製品名は、それを所有する各社の商標である場合があります。